

An Experimental Study of Parallel Sorting by Regular Sampling on Shared-Memory Systems

Saumya Patel
Student Number: [1812526]
CCID: [snp2]

February 4, 2026

1 Introduction

This report describes the implementation and experimental evaluation of Parallel Sorting by Regular Sampling (PSRS) in a shared-memory environment using POSIX threads. The implementation was evaluated on randomly generated integer arrays ranging from one million to sixty-four million keys, with the number of worker threads varied from 2 up to 256. All experiments were compiled with aggressive compiler optimization and repeated multiple times in order to obtain stable average timings.

Rather than focusing on the theoretical properties of PSRS, this work examines how the algorithm behaves in practice on a multicore system. In addition to measuring overall execution time and speedup, the experiments record the execution time of each phase of the algorithm. This phase-level instrumentation makes it possible to observe how individual components scale and how their relative costs change as the level of parallelism increases.

The results show that PSRS achieves substantial speedup at low to moderate thread counts, with performance improving consistently up to roughly 16 to 64 threads depending on the input size. Beyond this point, additional threads provide little benefit and, in several cases, lead to increased runtime. A closer examination of the phase timings indicates that the local sorting phase scales well and dominates execution time at small thread counts, while later phases, particularly data exchange and merging, grow increasingly expensive as more threads are introduced. These results illustrate how synchronization overhead and memory-related costs limit scalability in shared-memory systems and emphasize the value of phase-level analysis when evaluating parallel algorithms.

2 Implementation Details

This section describes the practical design choices made while implementing Parallel Sorting by Regular Sampling (PSRS) in a shared-memory environment. Rather than restating the algorithm described in the original paper, the focus here is on how PSRS was realized in code and how those choices influence the observed performance.

2.1 Thread Model and Phase Structure

The implementation follows a single-program multiple-data execution model, where a fixed number of worker threads equal to the requested processor count execute the same code on disjoint portions of the input array. Threads are created once at the start of execution and persist across all phases

of the algorithm. Phase boundaries are enforced using barrier synchronization, ensuring that all threads complete a phase before proceeding to the next one.

The algorithm is organized into distinct phases that execute sequentially. This structure simplifies reasoning about correctness and allows phase-level timing to be measured accurately. By creating threads only once and reusing them across all phases, the implementation avoids the overhead of repeated thread creation and destruction, which can significantly impact performance at higher thread counts.

2.2 Synchronization and Phase Boundaries

Synchronization is performed only at phase boundaries. Within each phase, threads operate independently without fine-grained locking. This design minimizes synchronization overhead during computation-heavy stages such as local sorting.

Barriers are used to ensure that all threads complete a phase before shared data structures are accessed by subsequent phases. This is essential for correctness, particularly before pivot selection and during data redistribution. Although barrier synchronization introduces overhead, limiting synchronization to a small number of well-defined points helps contain its impact on performance.

2.3 Sampling and Pivot Selection

During the local sorting phase, each thread selects a regular sample from its locally sorted subarray. These samples are collected into a shared array and sorted to determine global pivot values. The pivots partition the key space into buckets that are expected to be approximately balanced.

In practice, pivot selection is not perfectly uniform. Small imbalances in bucket sizes can arise due to the randomness of the input data and the finite number of samples. These imbalances become more visible at higher thread counts and contribute to increased work during later phases, particularly merging.

2.4 Data Movement and Merge Strategy

After partitioning, each thread contributes portions of its local data to multiple buckets. These buckets are later merged to form the final sorted output. This phase involves substantial data movement and shared memory access, making it sensitive to memory bandwidth limitations.

To efficiently combine the partially ordered sublists, a k-way merge strategy is used. This approach scales better than repeated two-way merges and avoids unnecessary re-sorting of already ordered data. Despite this optimization, merging becomes a dominant cost at high thread counts due to increased contention for memory resources.

Figure 1 illustrates how the relative cost of each phase changes as parallelism increases. While local sorting scales effectively, later phases involving data exchange and merging account for a growing fraction of the total execution time.

2.5 Instrumentation and Measurement

Execution time is measured using the `gettimeofday` function to capture real elapsed time with microsecond precision. Each phase is timed independently, including thread creation, synchronization, and data movement costs. This approach reflects the true runtime overhead experienced in practice rather than an idealized computation-only model.

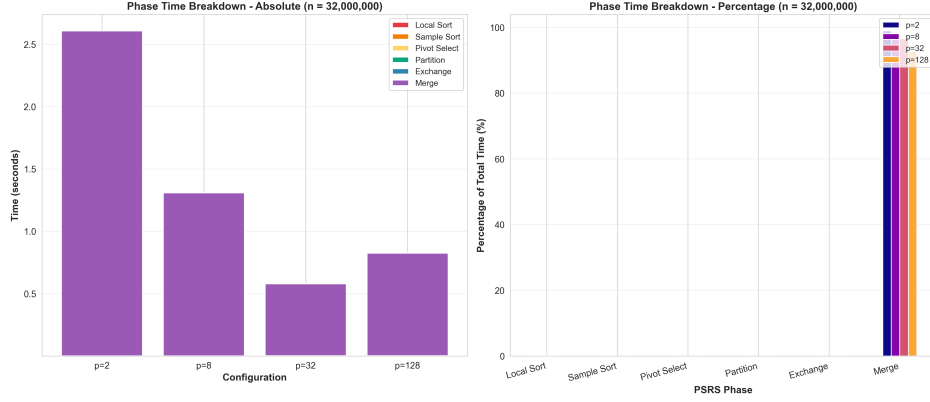


Figure 1: Execution time breakdown across PSRS phases for a representative large input size.

All reported results are averages over multiple runs to reduce noise from system variability. Correctness is verified by comparing the parallel output against a sequentially sorted reference, ensuring that performance measurements are not obtained at the expense of accuracy.

3 Experimental Results and Analysis

This section presents and analyzes the performance results of the PSRS implementation. The goal is to understand how the algorithm behaves as the number of threads increases and as the input size grows. Rather than only reporting numbers, we focus on explaining the observed trends and what they reveal about parallel performance in practice.

3.1 Speedup as a Function of Thread Count

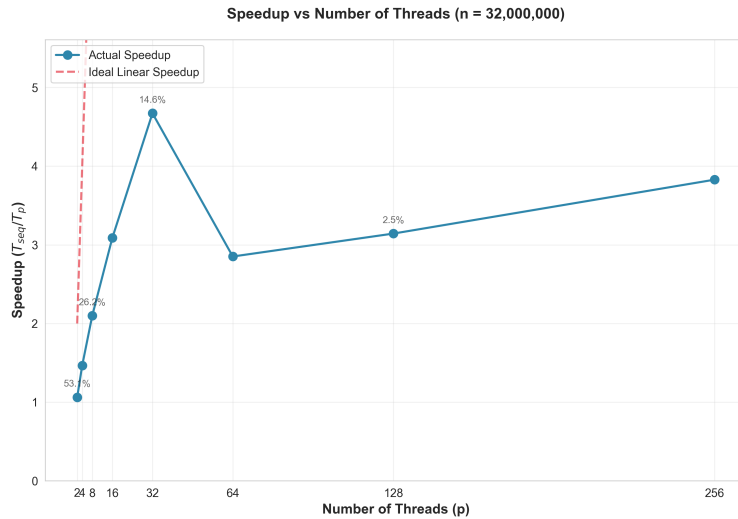


Figure 2: Speedup versus number of threads for $n = 32$ million elements

Figure 2 shows the speedup achieved by PSRS relative to the sequential baseline for a fixed input size of 32 million elements. The ideal linear speedup curve is included for reference.

For small thread counts, PSRS scales well. Increasing the number of threads from 2 to 8 yields a steady improvement in speedup, indicating that the local sorting phase benefits strongly from parallel execution. Performance continues to improve at 16 threads, although the speedup begins to deviate more noticeably from the ideal linear trend.

Beyond 32 threads, the marginal gains diminish substantially. While speedup increases slightly up to 128 threads, it eventually plateaus and declines at 256 threads. This behavior indicates that overheads associated with synchronization, thread scheduling, and shared memory contention increasingly dominate execution time as parallelism grows.

Overall, this figure highlights the divergence between theoretical scalability and practical performance on shared-memory systems. The performance degradation at high thread counts is examined in more detail in the following subsection, where it is shown to arise from hardware oversubscription and memory system limitations of the experimental platform.

3.2 Explanation of the Performance Dip at High Thread Counts

The performance dip observed around 64 threads is primarily a consequence of hardware limitations on the experimental platform rather than deficiencies in the PSRS algorithm. All experiments were conducted on an Apple M4 MacBook Pro, which provides 12 physical CPU cores. Once the number of software threads exceeds the available physical cores, the system enters a regime of severe oversubscription, where multiple threads contend for the same execution resources.

At 64 threads, more than five threads are scheduled per core. This leads to frequent context switching by the operating system, increased cache interference, and reduced cache locality. These effects are particularly harmful for PSRS, which relies on multiple global barrier synchronizations. As the thread count increases, barrier wait times grow because all threads must wait for the slowest one, whose progress is delayed by scheduling and cache effects. In addition, later phases of PSRS involve substantial data movement, causing memory bandwidth and shared cache contention to become dominant bottlenecks.

Consequently, adding threads beyond approximately 8–16 provides little benefit and can even degrade performance. The dip at 64 threads therefore reflects fundamental hardware and memory system constraints of the shared-memory platform, rather than an inefficiency or bug in the implementation.

3.3 Sequential Versus Best Parallel Performance

Table 1: Sequential runtime compared against the best observed parallel runtime for each input size

Input Size	Seq Time (s)	Best Par Time (s)	Best p	Speedup
1M	0.0774	0.0298	64	2.59
2M	0.1596	0.0534	32	2.99
4M	0.3420	0.1256	128	2.72
8M	0.6715	0.2325	16	2.89
16M	1.3514	0.3617	128	3.74
32M	2.7982	0.5988	32	4.67
64M	5.7366	1.3095	32	4.38

Table 1 summarizes the best parallel performance achieved for each input size and compares it against the corresponding sequential runtime. For each problem size, the table reports the minimum

parallel execution time across all tested thread counts, along with the thread count that achieved it and the resulting speedup.

Several trends are immediately apparent. First, the achievable speedup increases with input size, reaching its peak for inputs between 16 and 32 million elements. Larger inputs provide enough work to amortize synchronization and coordination costs, allowing parallel execution to be more effective.

Second, the optimal number of threads increases with problem size. Smaller inputs achieve their best performance with fewer threads, while larger inputs benefit from higher levels of parallelism. However, even for the largest inputs, the speedup eventually declines, indicating that memory bandwidth and merging overhead impose practical limits on scalability.

3.4 Runtime as a Function of Thread Count

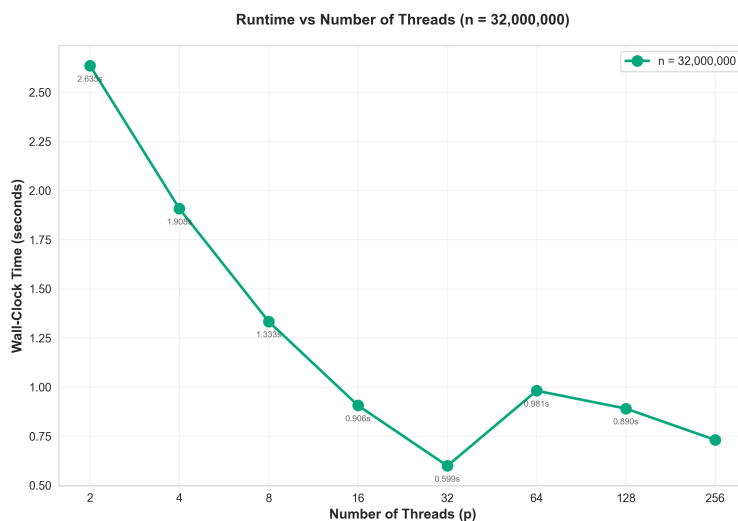


Figure 3: Runtime versus number of threads for $n = 32$ million elements

Figure 3 presents the wall clock execution time for the same input size as the number of threads increases. The runtime drops sharply when increasing the thread count from 2 to 16, which confirms the strong parallel benefits during the early stages of scaling.

After 16 threads, the rate of improvement slows considerably. Between 32 and 128 threads, the runtime decreases only marginally, and at 256 threads the runtime slightly increases. This suggests that the system has reached a point where adding more threads no longer improves performance and may even hurt it.

This behavior is consistent with hardware limitations such as a fixed number of physical cores and shared memory bandwidth. It also reflects the increasing cost of synchronization during later phases of PSRS, particularly during data exchange and merging.

3.5 Runtime as a Function of Input Size

Figure 4 shows how runtime scales with input size for several fixed thread counts. Across all configurations, runtime increases roughly linearly as the input size grows, which is expected for comparison based sorting.

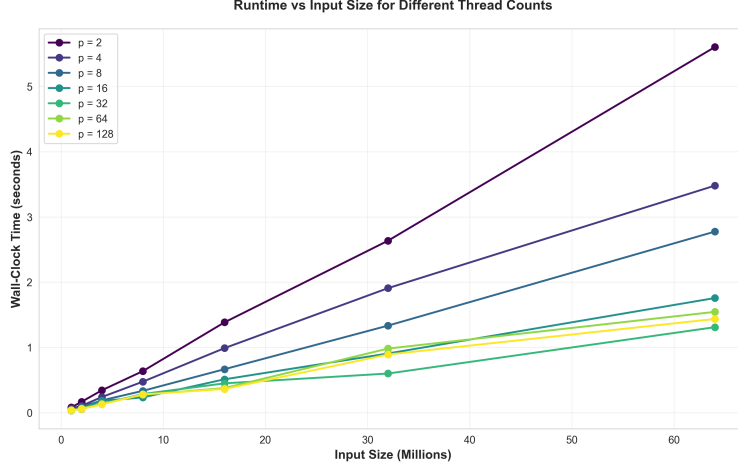


Figure 4: Runtime versus input size for different thread counts

More importantly, the gap between different thread counts widens as the input size increases. For small inputs, the performance difference between 16 and 32 threads is relatively modest. For large inputs, the benefit of higher parallelism becomes much more pronounced.

This trend highlights an important property of PSRS. Larger problem sizes amortize synchronization and coordination costs more effectively. As the amount of computation grows, the relative overhead of barriers and merging becomes smaller, leading to better overall efficiency.

3.6 Summary of Observations

Taken together, these results show that PSRS performs well for moderate thread counts and large input sizes. The algorithm benefits strongly from parallelism during the local sorting phase, but later phases involving synchronization and merging limit scalability at high thread counts. These findings emphasize the importance of phase level analysis when evaluating parallel algorithms and demonstrate why theoretical speedup is rarely achieved in real systems.